

Set Cover problem

Chosen solution

- List of sets

This way a random index can be generated to perform the desired tweak.

The starting point is a valid solution.

After it's generated, another list is initialized to track sets that are not in the solution.

```
# Solution as a list of sets
solution = []
while not isValid(solution):
    solution = sample(SETS, randint(0, N))
    available_sets = [s for s in SETS if s not in solution]
ic(available_sets)
```


Fitness function

- Opposite of the sum of the costs

If the solution is an empty one, the output will be the opposite of the sum of every single cost minus one. This was done to prevent getting a fitness of zero (which would be very high) if the starting solution is an empty list.

```
def fitness(solution: list[frozenset]) -> float:
    if len(solution) == 0:
        return -sum(COSTS) - 1
    return -sum(XCOSTS[s] for s in solution)
```

Chosen tweak

- Random adding, removal or swap of a set

Each operation has an 80% percent probability of being applied, thus allowing for a wider exploration of the solution space.

```
# Tweak
def tweak(solution: list[frozenset], sets: list[frozenset]):
    new_sol = solution.copy()
    new_available_sets = sets.copy()

    while random() < 0.8:
        operation = randint(0, 2)
        new_sol, new_available_sets = op_map[operation](new_sol, new_available_sets)

    return new_sol, new_available_sets
```

Chosen tweak

For each iteration, a random operation is applied:

- **Adding a set:** a random available set is popped and appended to the solution
- **Removing a set:** a random set is popped from the solution and appended to the available sets list
- **Swapping a set:** a random set from the solution is swapped with a random available set

```
# Tweak
```

```
def tweak(solution: list[frozenset], sets: list[frozenset]):  
    new_sol = solution.copy()  
    new_available_sets = sets.copy()  
  
    while random() < 0.8:  
        operation = randint(0, 2)  
        new_sol, new_available_sets = op_map[operation](new_sol, new_available_sets)  
  
    return new_sol, new_available_sets
```


Chosen tweak

Set removal and swapping are useful operation to avoid reaching a local maximum. The removal operation is especially useful to find smaller cover sets.

My initial implementation only allowed for swapping, so the solution was stuck with a fixed number of sets, and an optimal state couldn't have been reached.

```
# Tweak: swapping a set from the solution with one from the remaining available sets
def tweak(solution: list[frozenset], sets: list[frozenset]):
    rand_sol_idx = randint(0, SOL_SIZE - 1)
    rand_sets_idx = randint(0, len(sets) - 1)

    new_sol = solution.copy()
    new_available_sets = sets.copy()

    removed = new_sol.pop(rand_sol_idx)
    chosen = new_available_sets.pop(rand_sets_idx)

    new_sol.append(chosen)
    new_available_sets.append(removed)

    return new_sol, new_available_sets

def isValid(solution: list[frozenset]) -> bool:
    return frozenset.union(*solution) == frozenset(range(N))
```

Stopping criterion

- Amount of unsuccessful evolutions

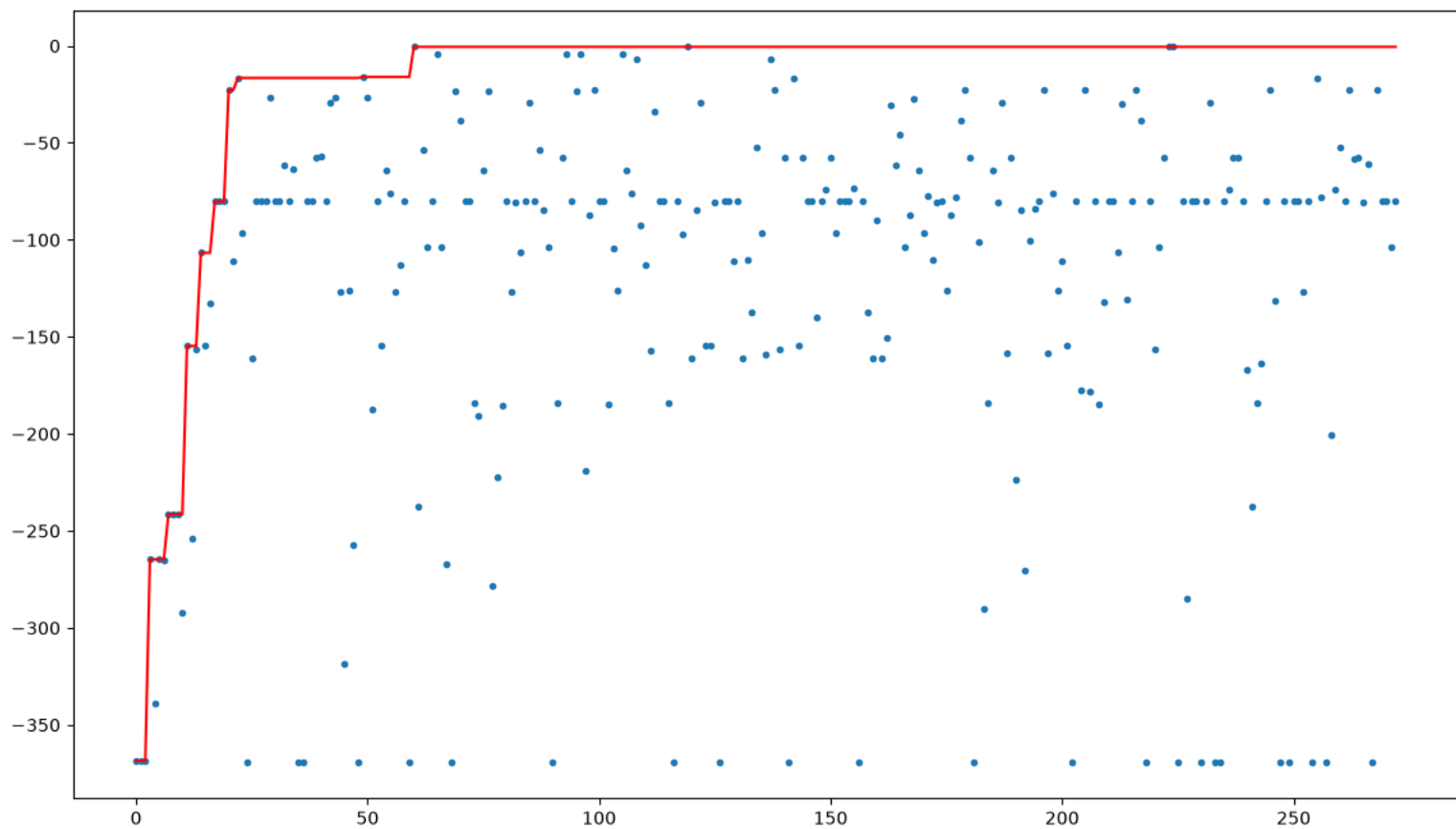
A variable is incremented if the solution produced by the tweak is either an illegal state or a state with a lower fitness than the previous solution. Once the threshold is reached, the algorithm stops.

The variable is resetted after a successful tweak

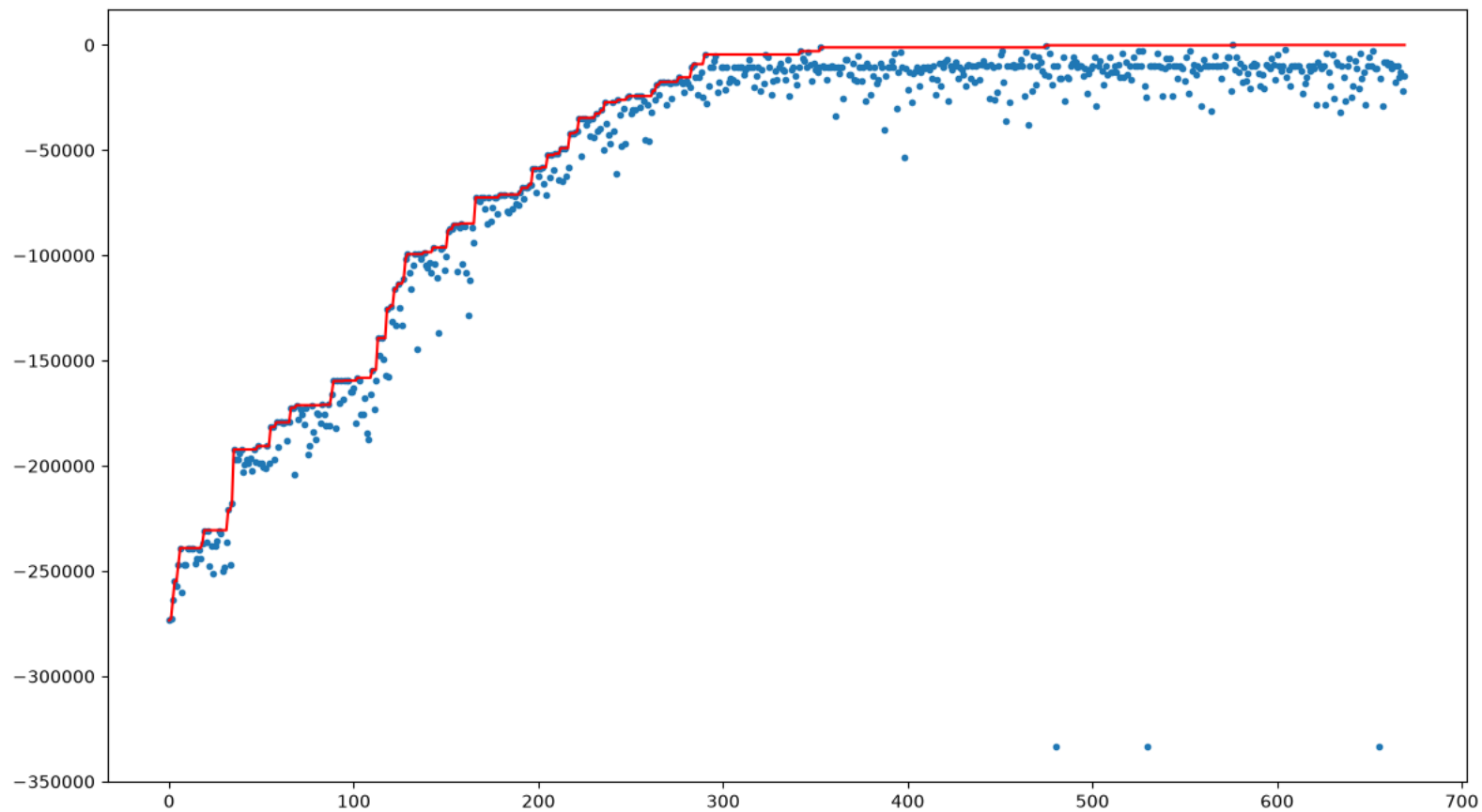
```
while step < MAX_STEPS:
    new_solution, new_sets = tweak(solution, available_sets)

    history.append(fitness(new_solution))
    if isValid(new_solution) and fitness(new_solution) > fitness(solution):
        solution = new_solution.copy()
        available_sets = new_sets.copy()
        step = 0
    else:
        step += 1
```

$N = 10$



$N = 100$



$N = 1000$

